Registro de Estudiantes con HTML, CSS, JavaScript y localStorage

David Raúl Sánchez Pincay

1. Objetivo

Crear una página web con HTML, CSS y JavaScript para el registro de estudiantes (cédula, apellidos, nombres, dirección, teléfono, correo electrónico, facultad, nivel y paralelo), con validaciones mediante expresiones regulares y almacenamiento local en formato JSON usando localStorage.

2. Estructura del proyecto

```
registro_estudiantes_localstorage/
 |-- index.html  (formulario, tabla y JSON almacenado)
 |-- style.css   (hoja de estilos externa)
 |-- script.js   (validaciones con regex y localStorage)
 `-- README.md
```

3. Repositorio Git

https://github.com/matsuro262/aplicaciones_cliente_web

La carpeta en la que se encuentra es registro_estudiantes_localstorage.

4. Campos y validaciones

Cada campo se valida con una expresión regular antes de guardar. Además, ningún campo puede quedar vacío.

Campo	Expresión regular	Qué valida
Cédula	`/^\d{10}$/`	Exactamente 10 dígitos.
Apellidos	`/^[A-Za-zÁÉÍÓÚÜÑáéíóúüñ]{2,}( [A-Za-zÁÉÍÓÚÜÑáéíóúüñ]{2,}){0,2}$/`	De 1 a 3 palabras solo con letras (mínimo 2 letras cada una).
Nombres	`/^[A-Za-zÁÉÍÓÚÜÑáéíóúüñ]{2,}( [A-Za-zÁÉÍÓÚÜÑáéíóúüñ]{2,}){0,2}$/`	De 1 a 3 palabras solo con letras (mínimo 2 letras cada una).
Dirección	`/^[A-Za-z0-9ÁÉÍÓÚÜÑáéíóúüñ .,#\-]{5,80}$/`	Entre 5 y 80 caracteres: letras, números, espacios y . , # -
Teléfono	`/^(09\d{8}|0[2-7]\d{7})$/`	Celular de 10 dígitos que empieza con 09, o convencional de 9 dígitos.
Correo electrónico	`/^[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-]+\.[A-Za-z]{2,}$/`	Formato usuario@dominio.extensión.
Facultad	`/^[A-Za-zÁÉÍÓÚÜÑáéíóúüñ ,]{5,60}$/`	Entre 5 y 60 letras, espacios y comas.

Campo	Expresión regular	Qué valida
Nivel	`/^([1-9]|10)$/`	Número del 1 al 10.
Paralelo	`/^[A-Za-z]$/`	Una sola letra.

5. Explicación del código HTML (index.html)

Todo el contenido está dentro de un <main id="contenedor"> que envuelve la página. Sus partes son:

- <form id="formEstudiante">: contiene los 9 campos. Cada campo es un div .campo con su <label>, su <input> o <select> y un donde se muestra el mensaje de validación. Tiene novalidate para que las validaciones las haga JavaScript.
- Cédula y teléfono usan maxlength="10" e inputmode="numeric"; paralelo usa maxlength="1"; nivel es un <select> con opciones del 1 al 10.
- Botones Registrar (submit) y Limpiar (reset), y un <p id="mensaje" role="status"> para los avisos.
- Una tabla <table> con 10 columnas; su <tbody id="tbodyEstudiantes"> se llena con JavaScript. Debajo está el botón Vaciar registros.
- Un <pre id="jsonAlmacenado"> que muestra el JSON guardado en localStorage.
- El script se enlaza al final del <body> con <script src="script.js">.

6. Código HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Registro de Estudiantes</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main id="contenedor">
    <h1>Registro de Estudiantes</h1>

    <!--FORMULARIO-->
    <form id="formEstudiante" novalidate>
      <div class="grid">
        <div class="campo">
          <label for="cedula">Cédula</label>
          <input type="text" id="cedula" maxlength="10" inputmode="numeric"
placeholder="1312345678">
```

```
          <span class="error" id="error-cedula"></span>
        </div>
        <div class="campo">
          <label for="apellidos">Apellidos</label>
          <input type="text" id="apellidos" placeholder="Pérez Loor">
          <span class="error" id="error-apellidos"></span>
        </div>
        <div class="campo">
          <label for="nombres">Nombres</label>
          <input type="text" id="nombres" placeholder="Juan Carlos">
          <span class="error" id="error-nombres"></span>
        </div>
        <div class="campo">
          <label for="direccion">Dirección</label>
          <input type="text" id="direccion" placeholder="Av. 4 de Noviembre y
Calle 12">
          <span class="error" id="error-direccion"></span>
        </div>
        <div class="campo">
          <label for="telefono">Teléfono</label>
          <input type="text" id="telefono" maxlength="10" inputmode="numeric"
placeholder="0991234567">
          <span class="error" id="error-telefono"></span>
        </div>
        <div class="campo">
          <label for="correo">Correo electrónico</label>
          <input type="text" id="correo" placeholder="juan.perez@correo.com">
          <span class="error" id="error-correo"></span>
        </div>
        <div class="campo">
          <label for="facultad">Facultad</label>
          <input type="text" id="facultad" placeholder="Ciencias de la Vida y
Tecnología">
          <span class="error" id="error-facultad"></span>
        </div>
        <div class="campo">
          <label for="nivel">Nivel</label>
          <select id="nivel">
            <option value="">Selecciona un nivel</option>
            <option value="1">1</option>
            <option value="2">2</option>
            <option value="3">3</option>
            <option value="4">4</option>
            <option value="5">5</option>
            <option value="6">6</option>
```

```
            <option value="7">7</option>
            <option value="8">8</option>
            <option value="9">9</option>
            <option value="10">10</option>
          </select>
          <span class="error" id="error-nivel"></span>
        </div>
        <div class="campo">
          <label for="paralelo">Paralelo</label>
          <input type="text" id="paralelo" maxlength="1" placeholder="A">
          <span class="error" id="error-paralelo"></span>
        </div>
      </div>

      <div class="botones">
        <button type="submit" class="btn btn-primario">Registrar</button>
        <button type="reset" class="btn btn-secundario">Limpiar</button>
      </div>
      <p id="mensaje" role="status" hidden></p>
    </form>

    <!--TABLA DE ESTUDIANTES-->
    <h2>Estudiantes registrados</h2>
    <div class="tabla-wrap" tabindex="0" role="region" aria-label="Tabla de
estudiantes registrados">
      <table>
        <thead>
          <tr>
            <th>Cédula</th>
            <th>Apellidos</th>
            <th>Nombres</th>
            <th>Dirección</th>
            <th>Teléfono</th>
            <th>Correo</th>
            <th>Facultad</th>
            <th>Nivel</th>
            <th>Paralelo</th>
            <th>Acción</th>
          </tr>
        </thead>
        <tbody id="tbodyEstudiantes"></tbody>
      </table>
    </div>
    <div class="botones">
```

```
      <button type="button" class="btn btn-secundario" id="btnVaciar">Vaciar
registros</button>
    </div>

    <!--JSON ALMACENADO-->
    <h2>JSON almacenado en localStorage</h2>
    <pre id="jsonAlmacenado" tabindex="0" aria-label="JSON almacenado en
localStorage"></pre>
  </main>

  <script src="script.js"></script>
</body>
</html>
```

7. Explicación de la hoja de estilos (style.css)

- Se enlaza mediante <link rel="stylesheet" href="style.css"> en el <head>.
- Selectores por id (#contenedor, #mensaje) y por clase (.campo, .error, .btn, .invalido, .ok, .fallo).
- .grid usa display: grid con auto-fit, por lo que los campos se acomodan en columnas según el ancho de la pantalla.
- Alto contraste: texto casi negro sobre fondo blanco, bordes de 2 px, mensajes de error en rojo oscuro y un anillo azul al enfocar (:focus).
- La tabla usa border-collapse, encabezado oscuro y filas alternas con nth-child(even); .tabla-wrap permite desplazarla en pantallas pequeñas.
- El <pre> usa la fuente Consolas, fondo gris claro y barras de desplazamiento para el JSON.

8. Código CSS

```
/*Hoja de estilos - Registro de Estudiantes*/
* { box-sizing: border-box; }

body {
  font-family: Arial, sans-serif;
  background-color: #dfe4ec;
  color: #0d0d0d;
  margin: 0;
}

#contenedor {
  max-width: 1000px;
  margin: 20px auto;
  background-color: #ffffff;
```

```
  border: 2px solid #1b2a4a;
  border-radius: 8px;
  padding: 24px;
}

h1, h2 {
  color: #1b2a4a;
  border-bottom: 3px solid #f4a300;
  padding-bottom: 4px;
}

/*Formulario*/
.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
  gap: 6px 20px;
}
.campo { display: flex; flex-direction: column; gap: 4px; margin-bottom: 8px; }
label { font-weight: bold; }

input, select {
  font: inherit;
  height: 42px;
  padding: 8px 10px;
  border: 2px solid #3d3d3d;
  border-radius: 6px;
  background-color: #ffffff;
  color: #0d0d0d;
}
::placeholder { color: #4a4a4a; opacity: 1; }
input:focus, select:focus, .btn:focus, .tabla-wrap:focus, pre:focus {
  outline: 3px solid #0a3cc2;
  outline-offset: 2px;
}
input.invalido, select.invalido {
  border-color: #9b1c1c;
  background-color: #fff0ee;
}
.error {
  color: #7a0e0e;
  font-size: 0.85em;
  font-weight: bold;
  min-height: 1.2em;
}
```

```css
/*Botones y mensajes*/
.botones { display: flex; gap: 10px; margin: 12px 0; }
.btn {
  font: inherit;
  font-weight: bold;
  padding: 8px 18px;
  border: 2px solid #1b2a4a;
  border-radius: 6px;
  cursor: pointer;
}
.btn-primario { background-color: #1b2a4a; color: #ffffff; }
.btn-secundario { background-color: #ffffff; color: #1b2a4a; }
.btn-eliminar { padding: 4px 10px; border-color: #9b1c1c; color: #7a0e0e;
background-color: #ffffff; }

#mensaje { padding: 10px 12px; border-radius: 6px; font-weight: bold; }
.ok { background-color: #d3ecd7; color: #0b3d14; border: 2px solid #14532d; }
.fallo { background-color: #fad4cf; color: #7a0e0e; border: 2px solid #9b1c1c; }

/*Tabla*/
.tabla-wrap { overflow-x: auto; border: 2px solid #1b2a4a; border-radius: 6px; }
table { width: 100%; min-width: 760px; border-collapse: collapse; }
th, td { padding: 6px 8px; text-align: left; border: 1px solid #6b6b6b; font-
size: 0.85em; }
td:nth-child(6) { overflow-wrap: anywhere; }
th:nth-child(1), td:nth-child(1), th:nth-child(5), td:nth-child(5),
th:nth-child(8), td:nth-child(8), th:nth-child(9), td:nth-child(9),
th:nth-child(10), td:nth-child(10) { white-space: nowrap; }
th { background-color: #1b2a4a; color: #ffffff; }
tbody tr:nth-child(even) { background-color: #f0f0f0; }
td.vacio { text-align: center; }

/*JSON almacenado*/
pre {
  font-family: Consolas, monospace;
  background-color: #f4f4f4;
  border: 2px solid #3d3d3d;
  border-radius: 6px;
  padding: 12px;
  max-height: 320px;
  overflow: auto;
}
```

9. Explicación del código JavaScript (script.js)

- REGEX es un objeto con una expresión regular por campo, y MENSAJES guarda el texto de error de cada una.
- validarCampo(id) toma el valor sin espacios (trim). Si está vacío muestra "Este campo es obligatorio."; si no, comprueba la expresión con test(). mostrarError() marca el campo en rojo y escribe el mensaje.
- Al enviar el formulario (evento submit) se validan los 9 campos. Si alguno falla, no se guarda nada y el foco va al primer campo con error. Cada campo también se valida al salir de él (evento focusout).
- Se evita repetir cédulas: si ya existe un estudiante con la misma, se muestra un error y no se registra.
- Almacenamiento: los estudiantes se guardan como un arreglo de objetos en formato JSON. guardarEstudiantes() usa JSON.stringify() y localStorage.setItem(); cargarEstudiantes() usa localStorage.getItem() y JSON.parse(), con try/catch por si el JSON estuviera dañado.
- mostrarEstudiantes() lee el JSON, llena la tabla (con textContent, para que el texto no se interprete como HTML) y muestra el JSON en el <pre>. Se ejecuta al abrir la página, por eso los datos siguen ahí después de recargar.
- Eliminar borra un estudiante por su cédula, y Vaciar registros usa localStorage.removeItem().

10. Código JavaScript

```
"use strict";

/*1. Expresiones regulares y mensajes de error*/
const REGEX = {
  cedula: /^\d{10}$/,
  apellidos: /^[A-Za-zÁÉÍÓÚÜÑáéíóúüñ]{2,}( [A-Za-zÁÉÍÓÚÜÑáéíóúüñ]{2,}){0,2}$/,
  nombres: /^[A-Za-zÁÉÍÓÚÜÑáéíóúüñ]{2,}( [A-Za-zÁÉÍÓÚÜÑáéíóúüñ]{2,}){0,2}$/,
  direccion: /^[A-Za-z0-9ÁÉÍÓÚÜÑáéíóúüñ .,#\-]{5,80}$/,
  telefono: /^(09\d{8}|0[2-7]\d{7})$/,
  correo: /^[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-]+\.[A-Za-z]{2,}$/,
  facultad: /^[A-Za-zÁÉÍÓÚÜÑáéíóúüñ ,]{5,60}$/,
  nivel: /^([1-9]|10)$/,
  paralelo: /^[A-Za-z]$/
};

const MENSAJES = {
  cedula: "La cédula debe tener exactamente 10 dígitos numéricos.",
  apellidos: "Ingresa de 1 a 3 apellidos, solo con letras (mínimo 2 letras cada
uno).",
```

```
  nombres: "Ingresa de 1 a 3 nombres, solo con letras (mínimo 2 letras cada
uno).",
  direccion: "La dirección debe tener entre 5 y 80 caracteres (letras, números,
espacios y . , # -).",
  telefono: "Ingresa un celular de 10 dígitos que empiece con 09, o un teléfono
convencional de 9 dígitos.",
  correo: "Ingresa un correo válido, por ejemplo nombre@correo.com.",
  facultad: "La facultad debe tener entre 5 y 60 letras.",
  nivel: "Selecciona un nivel.",
  paralelo: "El paralelo debe ser una sola letra (por ejemplo, A)."
};

const CAMPOS = Object.keys(REGEX);
const CLAVE = "estudiantes";
const form = document.getElementById("formEstudiante");

/*2. Validación de campos*/
function mostrarError(id, texto) {
  const input = document.getElementById(id);
  document.getElementById("error-" + id).textContent = texto;
  input.classList.toggle("invalido", texto !== "");
  input.setAttribute("aria-invalid", texto !== "" ? "true" : "false");
  input.setAttribute("aria-describedby", "error-" + id);
}

function validarCampo(id) {
  const valor = document.getElementById(id).value.trim();
  let texto = "";
  if (valor === "") texto = "Este campo es obligatorio.";
  else if (!REGEX[id].test(valor)) texto = MENSAJES[id];
  mostrarError(id, texto);
  return texto === "";
}

function mostrarMensaje(texto, correcto) {
  const m = document.getElementById("mensaje");
  m.textContent = texto;
  m.className = correcto ? "ok" : "fallo";
  m.hidden = false;
}

/*3. Almacenamiento local con JSON y localStorage*/
function cargarEstudiantes() {
  try {
    const datos = JSON.parse(localStorage.getItem(CLAVE));
```

```
    return Array.isArray(datos) ? datos : [];
  } catch (error) {
    return [];
  }
}

function guardarEstudiantes(lista) {
  localStorage.setItem(CLAVE, JSON.stringify(lista));
}

/*4. Mostrar la tabla y el JSON*/
function mostrarEstudiantes() {
  const lista = cargarEstudiantes();
  const tbody = document.getElementById("tbodyEstudiantes");
  tbody.innerHTML = "";

  if (lista.length === 0) {
    const fila = tbody.insertRow();
    const celda = fila.insertCell();
    celda.colSpan = 10;
    celda.className = "vacio";
    celda.textContent = "Aún no hay estudiantes registrados.";
  }

  lista.forEach((est) => {
    const fila = tbody.insertRow();
    CAMPOS.forEach((campo) => { fila.insertCell().textContent = est[campo]; });
    const boton = document.createElement("button");
    boton.type = "button";
    boton.className = "btn btn-eliminar";
    boton.textContent = "Eliminar";
    boton.dataset.cedula = est.cedula;
    boton.setAttribute("aria-label", "Eliminar al estudiante con cédula " +
est.cedula);
    fila.insertCell().appendChild(boton);
  });

  document.getElementById("jsonAlmacenado").textContent = JSON.stringify(lista,
null, 2);
}

/*5. Eventos*/
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const validos = CAMPOS.map(validarCampo);
```

```
  if (validos.includes(false)) {
    mostrarMensaje("Revisa los campos marcados en rojo.", false);
    form.querySelector('[aria-invalid="true"]').focus();
    return;
  }

  const estudiante = {};
  CAMPOS.forEach((id) => { estudiante[id] =
document.getElementById(id).value.trim(); });
  estudiante.paralelo = estudiante.paralelo.toUpperCase();

  const lista = cargarEstudiantes();
  if (lista.some((est) => est.cedula === estudiante.cedula)) {
    mostrarError("cedula", "Ya existe un estudiante registrado con esa cédula.");
    mostrarMensaje("No se registró: la cédula ya existe.", false);
    document.getElementById("cedula").focus();
    return;
  }

  try {
    lista.push(estudiante);
    guardarEstudiantes(lista);
  } catch (error) {
    mostrarMensaje("No se pudo guardar en el almacenamiento local.", false);
    return;
  }
  form.reset();
  mostrarMensaje("Estudiante registrado correctamente.", true);
  mostrarEstudiantes();
});

// Valida cada campo al salir de él
form.addEventListener("focusout", (e) => {
  if (CAMPOS.includes(e.target.id)) validarCampo(e.target.id);
});

// Al limpiar, se borran los errores y el mensaje
form.addEventListener("reset", () => {
  setTimeout(() => {
    CAMPOS.forEach((id) => mostrarError(id, ""));
    document.getElementById("mensaje").hidden = true;
  }, 0);
});

// Eliminar un estudiante
```

```
document.getElementById("tbodyEstudiantes").addEventListener("click", (e) => {
  const cedula = e.target.dataset.cedula;
  if (!cedula) return;
  guardarEstudiantes(cargarEstudiantes().filter((est) => est.cedula !== cedula));
  mostrarEstudiantes();
});

// Vaciar todos los registros
document.getElementById("btnVaciar").addEventListener("click", () => {
  if (confirm("¿Seguro que deseas eliminar todos los registros?")) {
    localStorage.removeItem(CLAVE);
    mostrarEstudiantes();
  }
});

/*6. Inicio*/
mostrarEstudiantes();
```

11. Ejecución - Resultado en el navegador

11.1 Formulario vacío

Al abrir la página la tabla está vacía y el JSON almacenado es un arreglo vacío ([]).



Figura 1. Formulario al abrir la página.

11.2 Validaciones

Se ingresaron datos incorrectos. Cada campo inválido se marca en rojo y muestra debajo su mensaje de error; no se guarda nada.

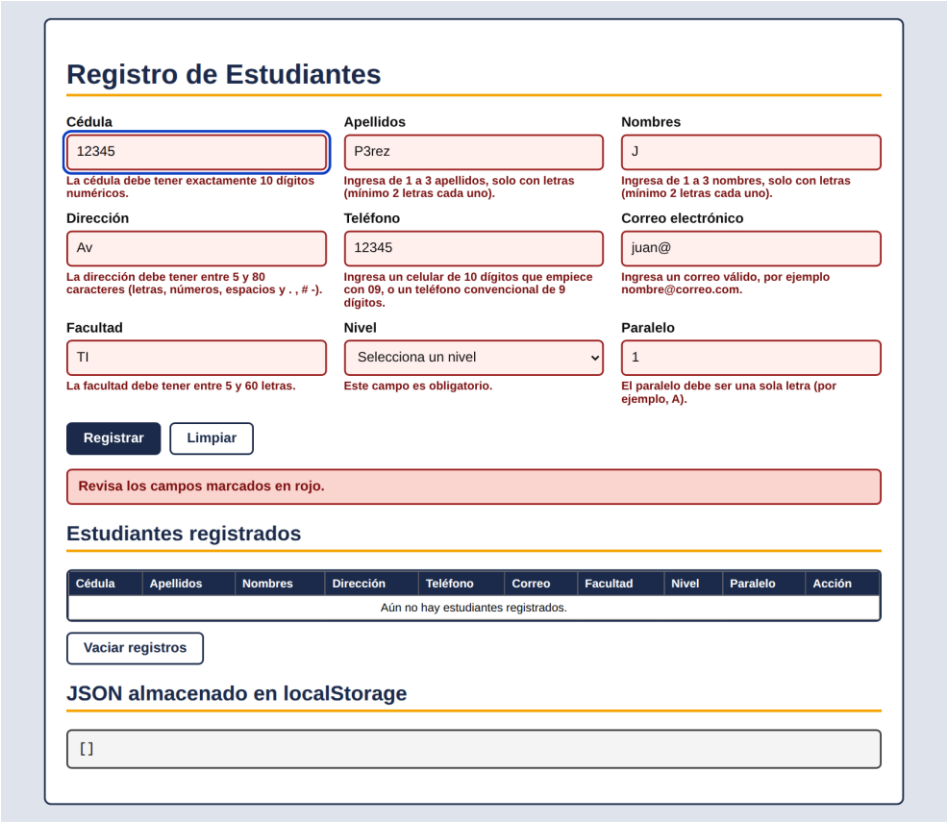


Figura 2. Validaciones con expresiones regulares.

11.3 Registro de estudiantes

Con datos válidos el estudiante se registra, la tabla se llena y el JSON muestra los objetos guardados en localStorage. El paralelo se guarda en mayúscula.

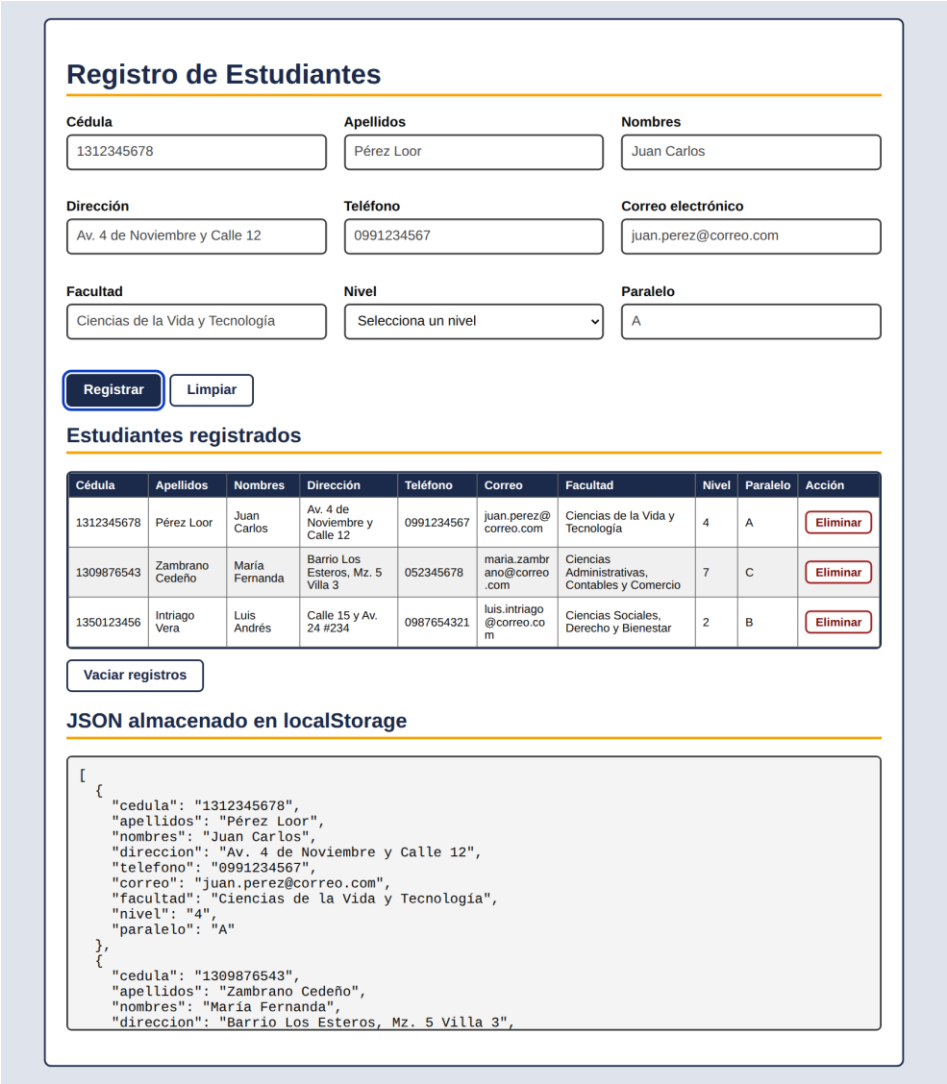


Figura 3. Estudiantes registrados y JSON almacenado.

11.4 Cédula duplicada

Si se intenta registrar una cédula que ya existe, la página lo rechaza y muestra el error.

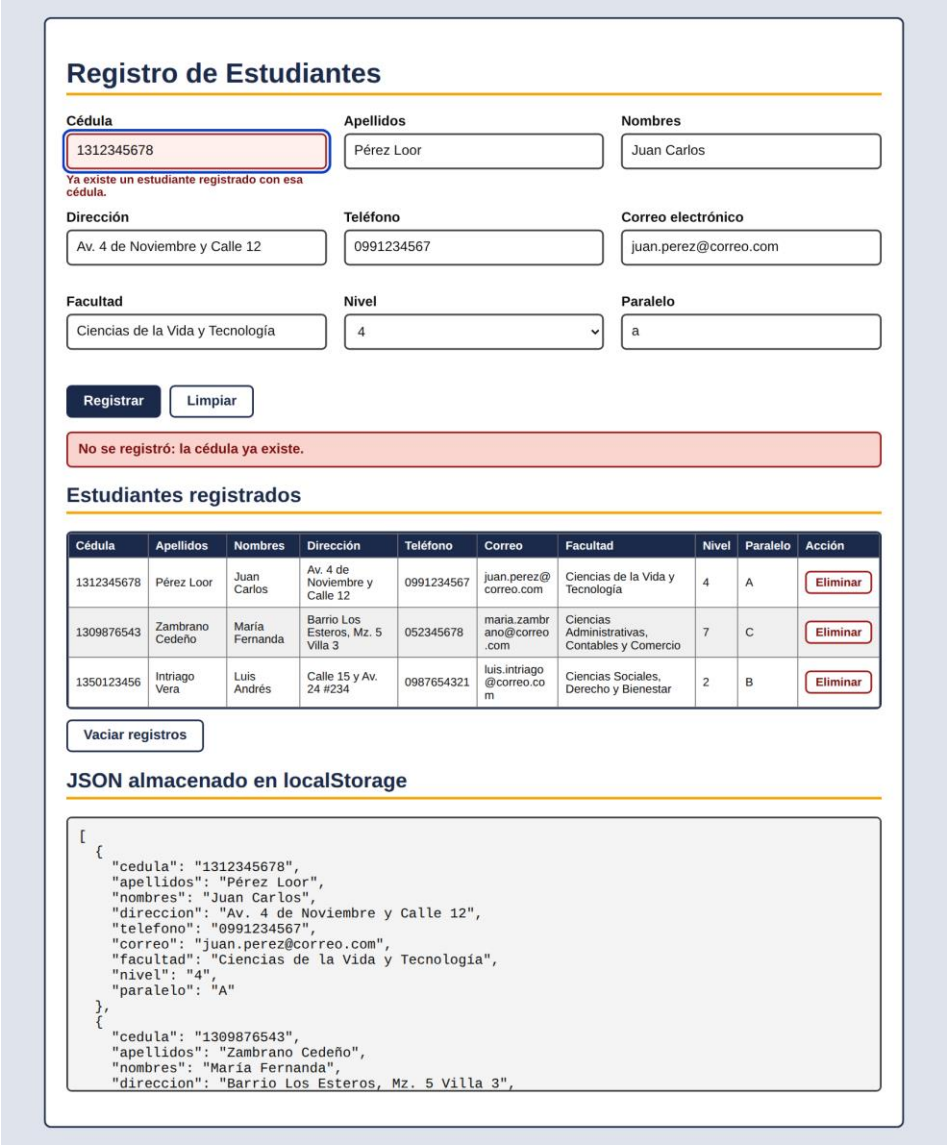


Figura 4. Validación de cédula duplicada.

11.5 Datos después de recargar

Al recargar la página los datos siguen ahí, porque están guardados en localStorage.

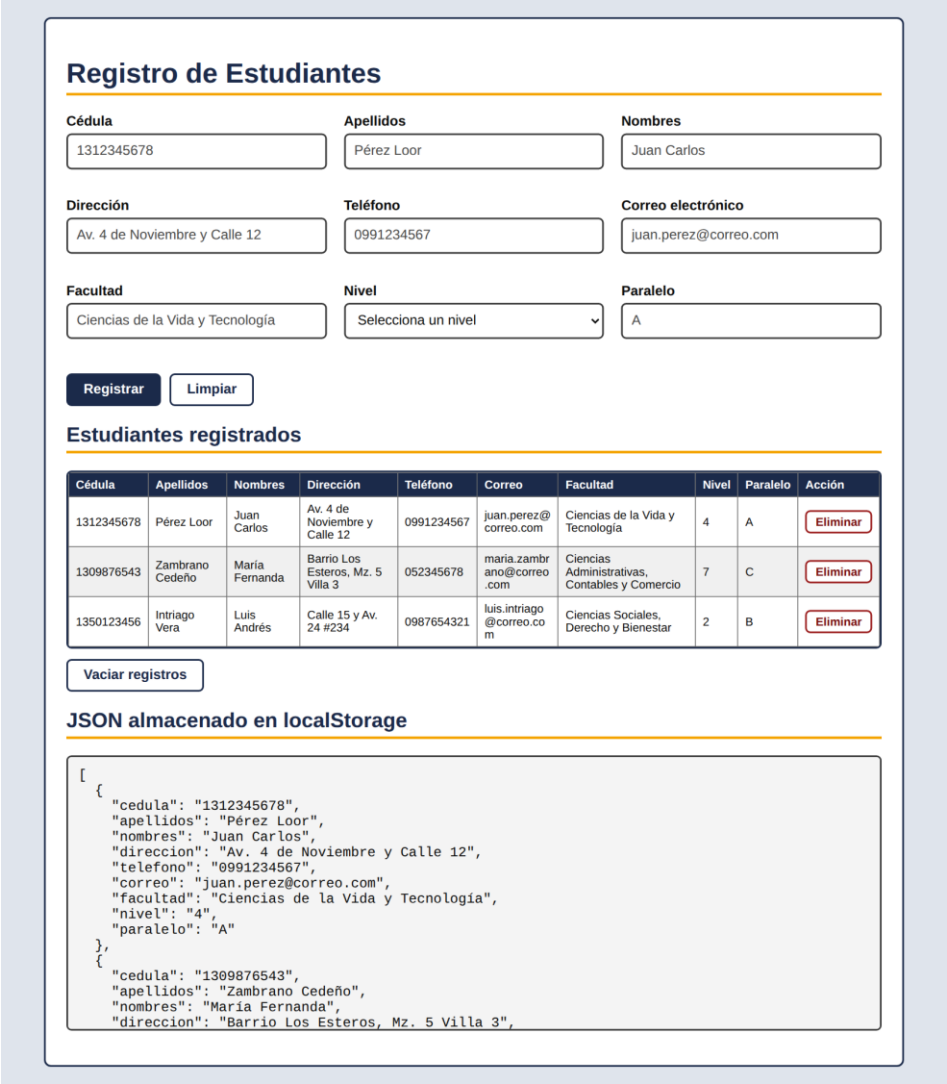


Figura 5. Página recargada con los datos guardados.